

Utilisation de Python dans *ArcGIS*

Table des matières

Prise en main des notebooks dans ArcGIS Pro	2
Configurer un projet	2
Créer un <i>notebook</i> et exécuter du code Python	3
Documenter un <i>notebook</i>	4
Exporter du code Python	4
Manipuler des classes d'entités	5
Exécuter une analyse à l'aide de Python	11
Exportation de code Python depuis l'historique	13
Références	14

Ce document comporte un ensemble d'exemples et d'exercices qui vous permettent d'apprendre à utiliser Python pour gérer, analyser et visualiser les données avec *ArcGIS Pro*.

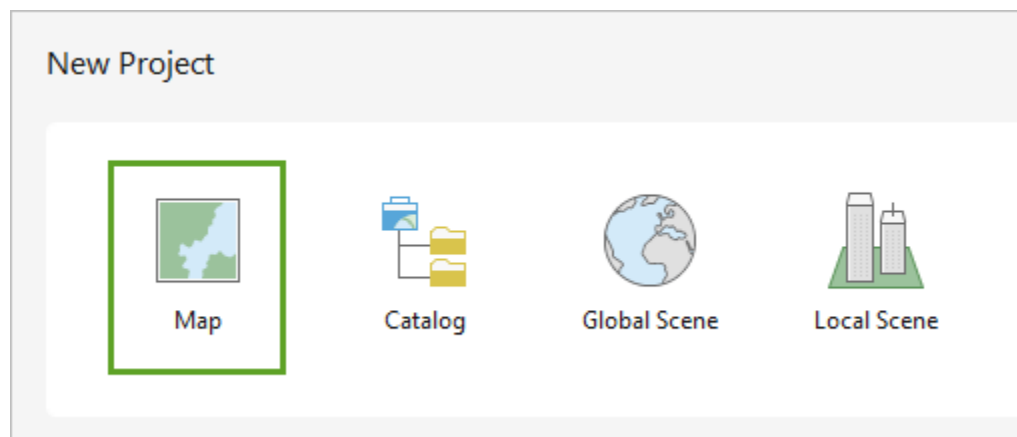
Prise en main des notebooks dans ArcGIS Pro

Les scripts Python peuvent être utilisés pour automatiser les processus dans *ArcGIS Pro*. Les *notebooks*, inclus dans *ArcGIS Pro*, constituent un environnement favorable pour écrire du code Python.

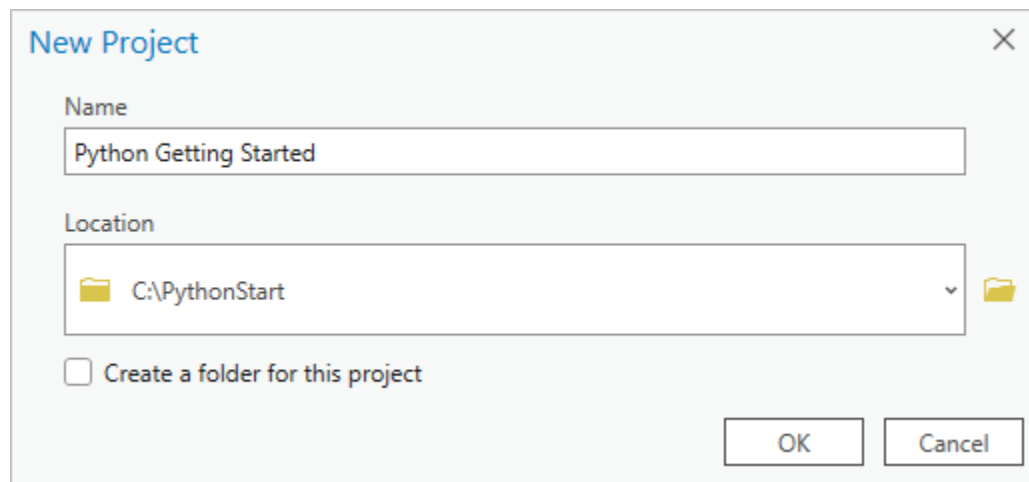
Les notebooks sont des environnements de programmation interactif basé sur le Web permettant de créer des documents *Jupyter Notebook*. Les documents *Jupyter Notebook* sont composés de: Texte, Markdown et code Python.

Configurer un projet

1. Démarrez *ArcGIS Pro*. Si vous y êtes invité, connectez-vous via votre compte ArcGIS.
2. Sous **New Project (Nouveau projet)**, cliquez sur **Map (Carte)**.



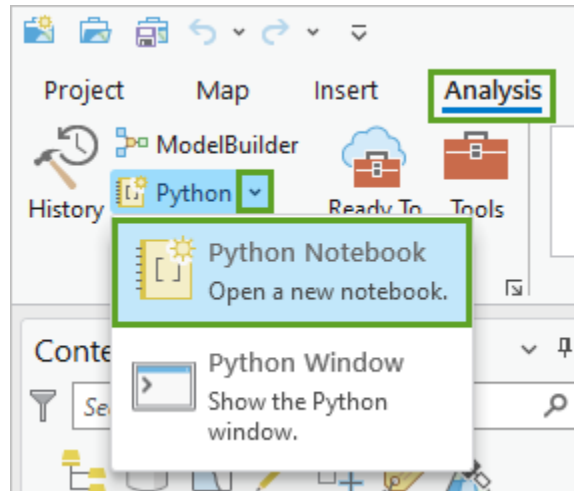
3. Dans la fenêtre **New Project (Nouveau projet)**, pour **Name (Nom)**, saisissez **Projet1**. Pour **Location (Emplacement)**, sélectionnez le dossier qui contiendra le projet.



4. Cliquez sur **OK**. Le projet s'ouvre avec une carte vierge.

Créer un *notebook* et exécuter du code Python

1. Sur le ruban, cliquez sur l'onglet **Analysis (Analyse)**, puis, dans le groupe **Geoprocessing (Géotraitement)**, cliquez sur la flèche du menu déroulant du bouton **Python** et cliquez sur **Python Notebook (Notebook Python)**.



Lorsque vous cliquez sur le bouton **Python** mais le menu déroulant vous permet de sélectionner un **notebook Python** ou une **fenêtre Python**. La **fenêtre Python** est une autre méthode pour exécuter du code Python dans *ArcGIS Pro*.

L'affichage du nouveau notebook peut prendre du temps et le message **Initializing Kernel (Initialisation du noyau)** peut s'afficher pendant que vous attendez. Ce message indique que *ArcGIS Pro* se prépare à exécuter le code du notebook. Le noyau est un logiciel qui s'exécute en arrière-plan pour exécuter le code Python saisi dans le notebook.

Une fois le notebook ouvert, il apparaît sous la forme d'une nouvelle vue dans la fenêtre principale de *ArcGIS Pro*.

Le nouveau notebook est stocké en tant que fichier **.ipnyb** dans le dossier d'accueil de votre projet. Le nouveau notebook s'affiche également dans le dossier **Notebooks** de la fenêtre **Catalog (Catalogue)**.

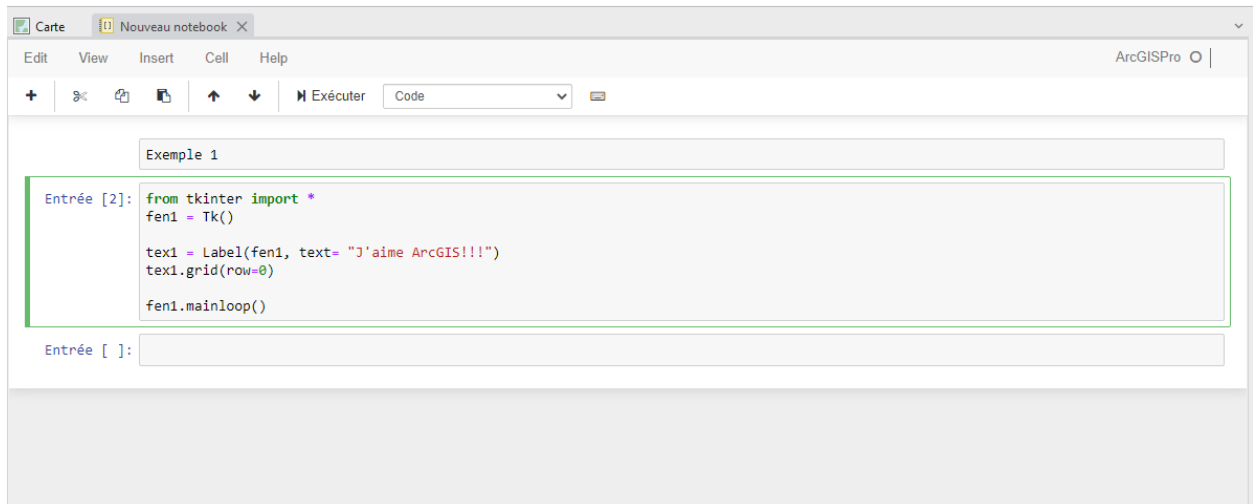
Exemple 1

Saisissez les lignes de code suivantes:

```

1 from tkinter import *
2 fen1 = Tk()
3
4 tex1 = Label(fen1, text= "J'aime ArcGIS!!!")
5 tex1.grid(row=0)
6
7 fen1.mainloop()

```



Dans la barre d'outils au-dessus de la cellule, cliquez sur le bouton **Run (Exécuter)**.

Documenter un *notebook*

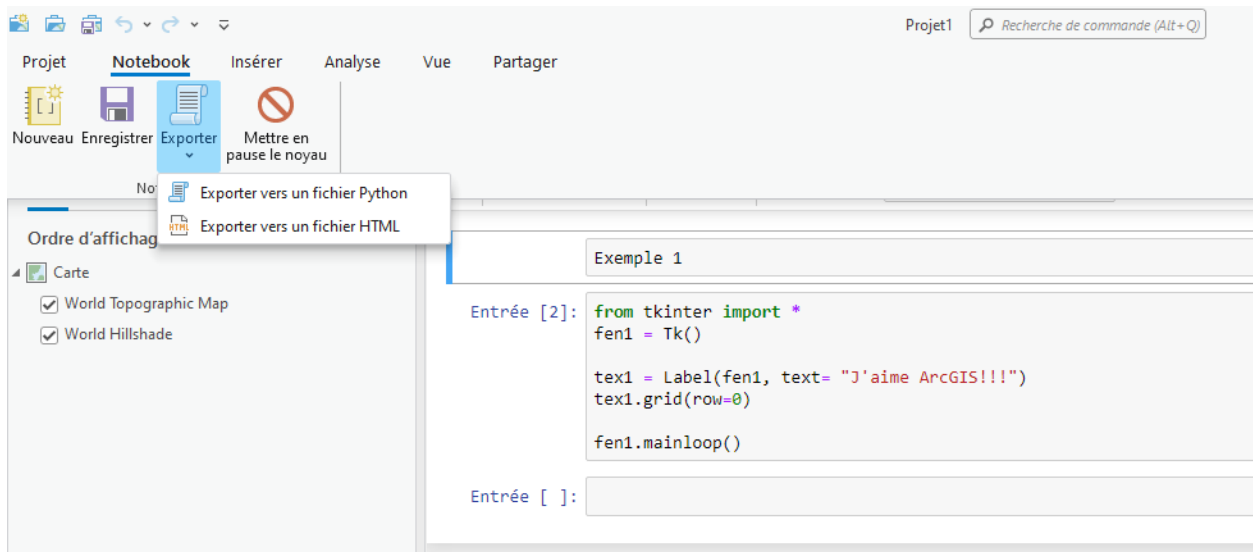
La prise de note dans un *notebook* peut se faire avec du *Markdown*. Le *Markdown* est un langage de balisage léger créé dans le but d'offrir une syntaxe facile à lire et à écrire dans sa forme non formatée. *Markdown* est principalement utilisé dans les blogs, les sites de messagerie instantanée, les forums et les pages de documentation de logiciels.

Élément	Syntaxe markdown
Titre	# Titre
Sous-titre	## Titre
Gras	** texte **
Italique	<i>* texte *</i>
Liste	1. premier élément
Puces	- premier élément

Exporter du code Python

Il est possible d'exporter du code Python à partir d'un *notebook*. Pour ce faire, sélectionnez

Notebook sur le ruban, puis cliquez sur **Exporter**.

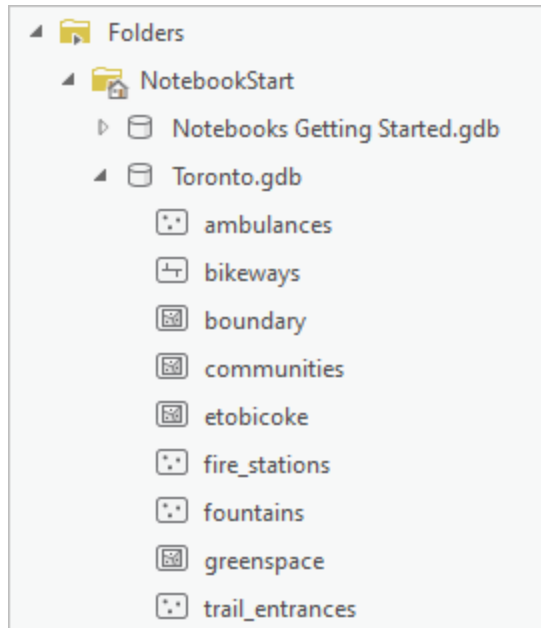


Manipuler des classes d'entités

1. Téléchargez les données de ce didacticiel et extrayez le contenu dans un emplacement sur votre ordinateur. Le fichier .zip contient un dossier nommé **NotebookStart**.
2. Cliquez sur **Open another project (Ouvrir un autre projet)**.
3. Dans la fenêtre **Open Project (Ouvrir le projet)**, accédez au dossier que vous avez extrait du fichier **NotebookStart.zip**, cliquez sur le fichier **Notebooks Getting Started.aprx** pour le sélectionner, puis cliquez sur **OK**.

Le projet s'ouvre. La carte actuelle montre la limite de la ville de Toronto, au Canada. Vous allez ajouter des classes d'entités à cette carte.

4. Dans la fenêtre **Catalog (Catalogue)**, développez **Folders (Dossiers)**, puis développez **NotebookStart**.
5. Développez la base de données géoréférencées **Toronto.gdb**.



La base de données géoréférencées contient plusieurs classes d'entités (**tables**).

ATTENTION!

Une classe d'entités est un ensemble d'entités géographiques qui partagent le même type de géométrie (par exemple, point, ligne ou polygone) et les mêmes champs attributaires pour une zone commune. En informatique, le terme utilisé pour identifier une classe d'entités est **table**.

6. Cliquez avec le bouton droit sur la classe d'entités **etobicoke**, puis cliquez sur **Add To Current Map (Ajouter à la carte actuelle)**.

Ces polygones représentent plusieurs communautés au sein de la ville de Toronto.

7. Ajoutez les classes d'entités **fire_stations** et **greenspace** à la carte.

Ces classes d'entités représentent les emplacements des casernes de pompiers et des espaces verts de la ville de Toronto, respectivement. Vous allez utiliser ces classes d'entités pour exécuter divers outils de géotraitement à l'aide de Python

8. Sur le ruban, cliquez sur l'onglet **Analysis (Analyse)**, puis, dans le groupe **Geoprocessing (Géotraitement)**, cliquez sur la flèche du menu déroulant du bouton Python et cliquez sur **Python Notebook (Notebook Python)**.

Exemple 2

Saisissez les lignes de code suivantes:

```
1 import arcpy
2 arcpy.GetCount_management("fire_stations")
```

Dans la barre d'outils au-dessus de la cellule, cliquez sur le bouton **Run (Exécuter)**.

```
Exemple 2

Entrée [1]: import arcpy
arcpy.GetCount_management("fire_stations")

Out[1]:
Messages
Heure de début : 25 avril 2024 07:31:29
Nombre de lignes = 84
réussie à 25 avril 2024 07:31:29 (temps écoulé : 0,11 secondes)
```

La fonction `arcpy.GetCount_management()` permet d'afficher le nombre d'éléments (d'enregistrements) contenu dans une classe d'entités.

Exemple 3

Saisissez les lignes de code suivantes:

```
1 import arcpy
2
3 arcpy.env.workspace = "C:/Users/Profs/Desktop/NotebookStart/Toronto.gdb"
4 fc_list = arcpy.ListFeatureClasses()
5 print(fc_list)
```

L'instruction `arcpy.env.workspace` (ligne 3) permet d'identifier la base de données géoréférencées qui sera utilisée. À la ligne 4, la fonction `arcpy.ListFeatureClasses()` permet de créer une liste des noms des classes d'entités contenu dans la base de données.

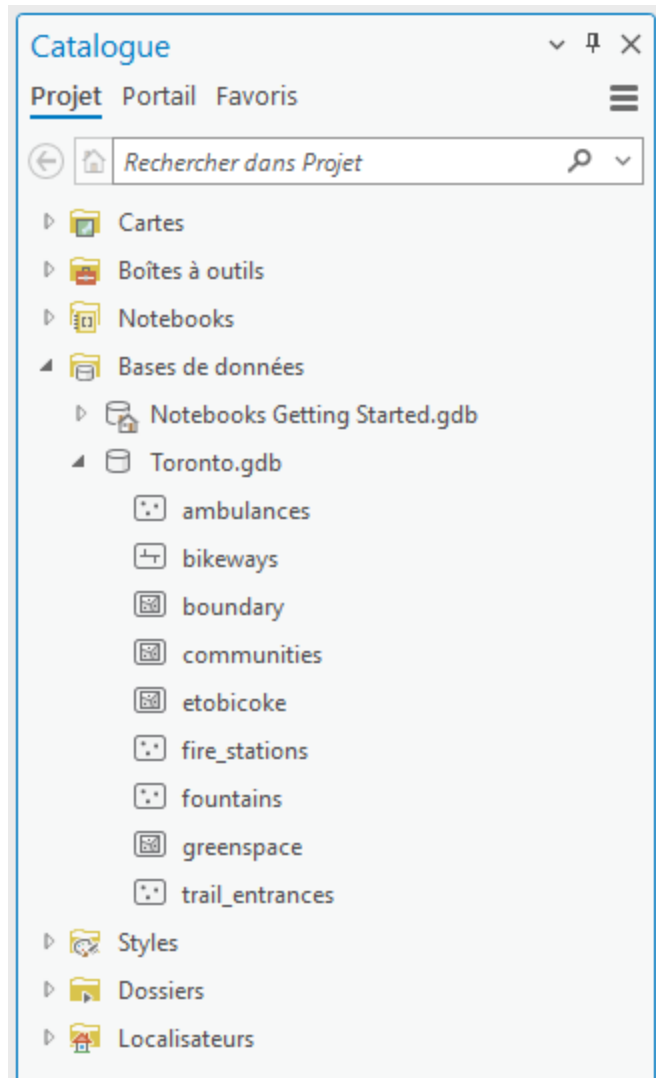
```
Exemple 3

Entrée [2]: import arcpy

arcpy.env.workspace = "C:/Users/Profs/Desktop/NotebookStart/Toronto.gdb"
fc_list = arcpy.ListFeatureClasses()
print(fc_list)

['trail_entrances', 'boundary', 'fountains', 'fire_stations', 'ambulances', 'communities', 'greenspace', 'etobicoke', 'bikeways']
```

Comparez le résultat obtenu avec le contenu de la base de données affichée dans la fenêtre **Catalog**.



Exemple 4

Saisissez les lignes de code suivantes:

```
1 import arcpy
2
3 fields = arcpy.ListFields("fire_stations")
4
5 for field in fields:
6     print(f"{field.name} \t{field.type} \t{field.length}")
```

La fonction `arcpy.ListFields()` permet de créer la liste des champs d'une classe d'entités.


```
Entrée [5]: import arcpy

fields = arcpy.ListFields("fire_stations")

for field in fields:
    print(f"{field.name}\t{field.type}\t{field.length}")

OBJECTID_1      OID      4
Shape           Geometry  0
NAME            String   100
ADDRESS         String   100
WARD_NAME       String   40
MUN_NAME        String   15
```

Afficher le contenu de la classe d'entité *fire_stations*.

OBJECTID_1	Shape	NAME	ADDRESS	WARD_NAME	MUN_NAME
1	Point	FIRE STATION 214	745 MEADOWVALE RD	Scarborough East (44)	Scarborough
2	Point	FIRE STATION 215	5318 LAWRENCE AVE E	Scarborough East (44)	Scarborough
3	Point	FIRE STATION 221	2575 EGLINTON AVE E	Scarborough Southwe...	Scarborough
4	Point	FIRE STATION 222	755 WARDEN AVE	Scarborough Southwe...	Scarborough
5	Point	FIRE STATION 223	116 DORSET RD	Scarborough Southwe...	Scarborough
6	Point	FIRE STATION 224	1313 WOODBINE AVE	Beaches-East York (31)	East York
7	Point	FIRE STATION 225	3600 DANFORTH AVE	Scarborough Southwe...	Scarborough
8	Point	FIRE STATION 226	87 MAIN ST	Beaches-East York (32)	former Toronto
9	Point	FIRE STATION 227	1904 QUEEN ST E	Beaches-East York (32)	former Toronto
10	Point	FIRE STATION 231	740 MARKHAM RD	Scarborough Centre (38)	Scarborough

Exemple 5

Saisissez les lignes de code suivantes:

```
1 import arcpy
2
3 fields = ["NAME", "ADDRESS"]
4
5 for ligne in arcpy.da.SearchCursor("fire_stations", fields):
6     print(ligne[0], "\t", ligne[1])
```

Dans cette exemple une partie du contenu d'une classe d'entités est affiché.

Exemple 5

```
Entrée [6]: import arcpy

fields = ["NAME", "ADDRESS"]

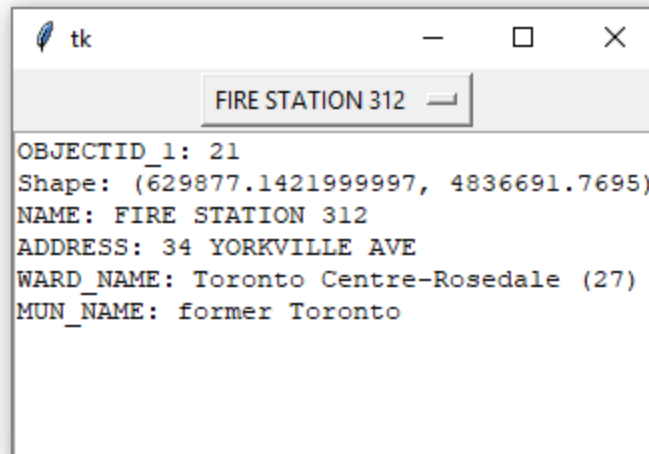
for ligne in arcpy.da.SearchCursor("fire_stations", fields):
    print(ligne[0], "\t", ligne[1])

FIRE STATION 214      745 MEADOWVALE RD
FIRE STATION 215      5318 LAWRENCE AVE E
FIRE STATION 221      2575 EGLINTON AVE E
FIRE STATION 222      755 WARDEN AVE
FIRE STATION 223      116 DORSET RD
FIRE STATION 224      1313 WOODBINE AVE
FIRE STATION 225      3600 DANFORTH AVE
FIRE STATION 226      87 MAIN ST
FIRE STATION 227      1904 QUEEN ST E
FIRE STATION 231      740 MARKHAM RD
FIRE STATION 232      1550 MIDLAND AVE
FIRE STATION 233      59 CURLEW DR
FIRE STATION 234      40 CORONATION DR
FIRE STATION 235      700 RERMONDSEY RD
```

Exemple 6

Créez un script qui permet d'afficher les informations associées à une caserne. L'utilisateur doit saisir le nom d'une caserne à l'aide d'une liste déroulante.

```
1 from tkinter import *
2 import arcpy
3
4 arcpy.env.workspace = "C:/Users/Profs/Desktop/NotebookStart/Toronto.gdb"
5 fields = ["NAME"]
6 liste_casernes=[]
7 for ligne in arcpy.da.SearchCursor("fire_stations", fields):
8     liste_casernes.append(ligne[0])
9
10 liste_champs = []
11 fields = arcpy.ListFields("fire_stations")
12 for field in fields:
13     liste_champs.append(field.name)
14
15
16 def detail_caserne(event):
17     caserne = choix.get()
18     #fields = ["NAME"]
19     resultat_recherche=[]
20     qry = " NAME = '" + caserne + "' "
21     for ligne in arcpy.da.SearchCursor("fire_stations", liste_champs, qry):
22         #print(ligne[0])
23         for index in range(len(liste_champs)):
24             info.insert(INSERT, str(liste_champs[index] + ": " + str(ligne
25 [index])) + "\n")
26
27 fen1 = Tk()
28
29 choix = StringVar(fen1)
30 choix.set(liste_casernes[0])
31 liste = OptionMenu(fen1, choix, *liste_casernes, command=detail_caserne)
32 liste.grid(row=0)
33
34 info = Text(fen1)
35 info.config(width=40, height=10)
36 info.grid(row=1)
37
38 fen1.mainloop()
```



Exercice 1

Affichez la liste des casernes qui se trouve dans la municipalité de Scarborough.

Exécuter une analyse à l'aide de Python

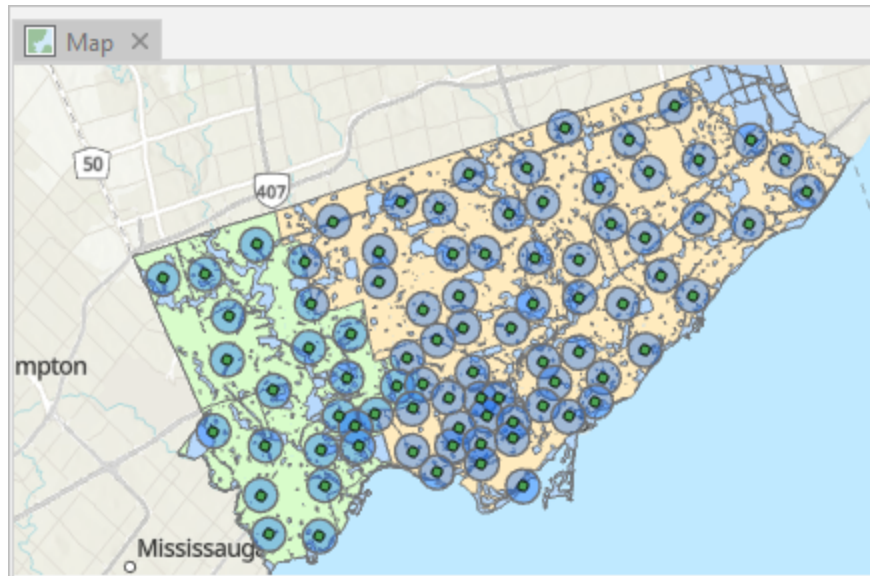
Vous allez ensuite procéder à des analyses SIG à l'aide du notebook. Supposons que vous souhaitiez identifier les zones du district administratif d'Etobicoke qui sont les plus éloignées des casernes de pompiers. Pour ce faire, vous pouvez utiliser des outils de géotraitement dans un notebook.

Exemple 7

Saisissez les lignes de code suivantes:

```
1 import arcpy
2
3 arcpy.Buffer_analysis("fire_stations", "fire_buffer", "1000 METERS")
```

Dans la barre d'outils au-dessus de la cellule, cliquez sur le bouton **Run (Exécuter)**. Observez la carte. Les résultats montrent les zones situées à 1 000 mètres (1 kilomètre) ou moins d'une caserne de pompiers, et celles qui ne le sont pas.



La fonction `arcpy.Buffer_analysis()` permet de calculer une zone tampon d'un rayon spécifié par le paramètre `"1000 METERS"`. Le paramètre `"fire_stations"` est la classe d'entité qui sera utilisé pour l'analyse. La classe d'entité en sortie est spécifiée par le deuxième paramètre `"fire_buffer"`

Exemple 8

Saisissez les lignes de code suivantes:

```
1 import arcpy
2
3 arcpy.env.overwriteOutput = True
4 arcpy.Buffer_analysis("fire_stations", "fire_buffer", "1750 METERS")
```

Dans la barre d'outils au-dessus de la cellule, cliquez sur le bouton **Run (Exécuter)**. Observez la carte.

L'instruction `arcpy.env.overwriteOutput` permet d'écraser les résultats de l'exécution précédente.

Exemple 9

Ajoutez la classe d'entité `etobicoke` à la carte puis, saisissez les lignes de code suivantes:

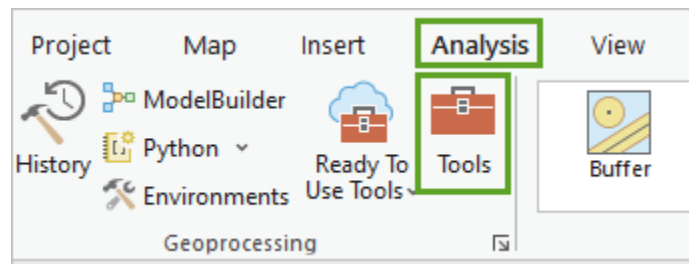
```
1 import arcpy
2
3 arcpy.env.overwriteOutput = True
4 arcpy.Buffer_analysis("fire_stations", "fire_buffer", "1750 METERS")
5 arcpy.PairwiseErase_analysis("etobicoke", "fire_buffer", "no_service")
```

Dans la barre d'outils au-dessus de la cellule, cliquez sur le bouton **Run (Exécuter)**. Observez la carte.

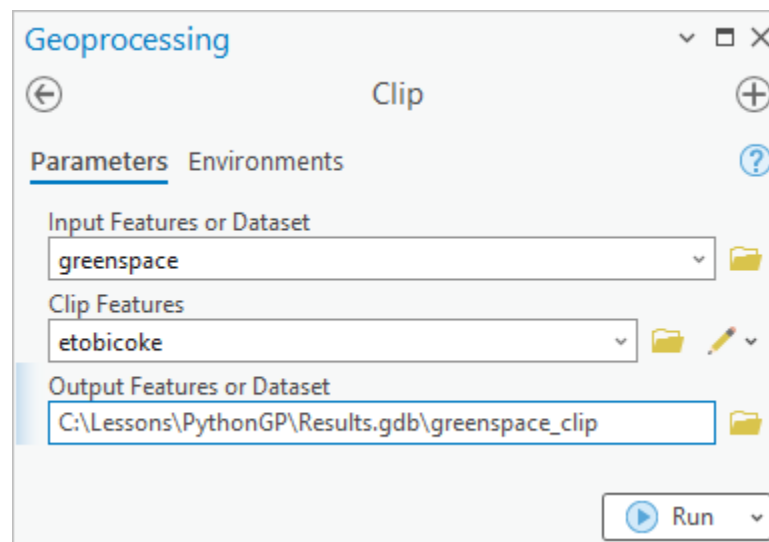
La fonction `PairwiseErase_analysis()` sur la classe d'entités "`etobicoke`", en efface les secteurs de la classe d'entités "`fire_buffer`", puis écrit les résultats dans une nouvelle classe d'entités nommée "`no_service`".

Exportation de code Python depuis l'historique

1. Dans la fenêtre **Catalog (Catalogue)**, accédez à la géodatabase **Toronto.gdb**.
2. Ajoutez les classes d'entités **greenspace (greenspace)** et **etobicoke (etobicoke)** à la carte active.
3. Sur le ruban, cliquez sur **Analysis (Analyse)**, puis, dans le groupe **Geoprocessing (Géotraitement)**, cliquez sur **Tools (Outils)**.



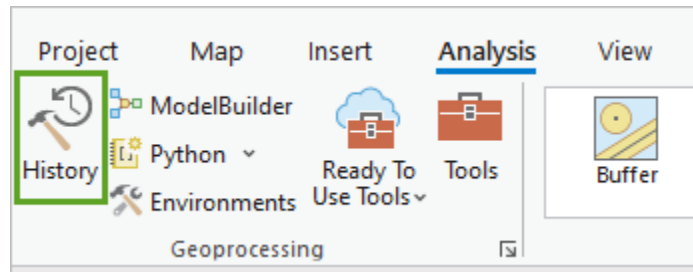
4. Pour les **Input Features (Entités en entrée)**, sélectionnez **greenspace (greenspace)**.
5. Pour les **Clip Features (Entités de découpage)**, sélectionnez **etobicoke (etobicoke)**.
6. Pour **Output Features or Dataset (Entités ou jeu de données en sortie)**, accédez à **Results.gdb** et nommez la classe d'entités en sortie **greenspace_clip**.



7. Cliquez sur **Run (Exécuter)**.

L'outil **Clip (Découper)** est exécuté et la classe d'entités résultante est ajoutée à la carte.

8. Sur le ruban, dans le groupe **Geoprocessing (Géotraitement)**, cliquez sur **History (Historique)**.



La fenêtre **Geoprocessing History** (Historique de géotraitement) apparaît.

9. Dans la liste **Geoprocessing History (Historique de géotraitement)**, cliquez avec le bouton droit sur le traitement à exporter et sélectionnez **Send To Python window (Envoyer vers la fenêtre Python)**.

```
arcpy.analysis.Clip("greenspace", "etobicoke", r"C:\Lessons\PythonGP\Results.gdb  
greenspace_clip", None)
```

Références

- Découvrir Python dans ArcGIS Pro
- Three Minutes to Your First Python Script
- Esri Training Videos